Sprint 3

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Grupo 2:

Alejandro Alcázar Mendoza
Sergio Fernández Cordero
Liam Esgueva González

Índice

1. Introducción **3**
1.1. Contexto . . . 3
1.2. Descripción del problema y objetivos del sprint. . . . 3
1.3. Justificación . . . 3

2. Metodología **4**
2.1. Entorno de desarrollo . . . 4
2.2. Diseño de la solución . . . 4
2.3. Metodología SCRUM . . . 5

3. Implementación **6**
3.1. Arquitectura del pipeline ETL . . . 6
3.2. Procesamiento con Apache Spark: Bronze to Silver . . . 6
3.2.1. Inicialización del entorno Spark . . . 6
3.2.2. Transformaciones con Spark SQL . . . 7
3.2.3. Optimizaciones de Spark aplicadas . . . 7
3.2.4. Escritura optimizada en Parquet . . . 8
3.3. Ventajas del procesamiento con Spark . . . 8
3.4. Procesamiento con Apache Spark: Silver to Gold . . . 8
3.4.1. Arquitectura distribuida con Window Functions . . . 8
3.4.2. Cálculo de indicadores técnicos . . . 9
3.4.3. Gestión del warm-up period . . . 12
3.4.4. Optimizaciones de Spark aplicadas . . . 13
3.4.5. Persistencia en capa Gold . . . 13
3.4.6. Resumen de indicadores calculados . . . 14

4. Resultados **14**
4.1. Descripción . . . 14
4.2. Ejecución . . . 14
4.3. Discusión de los resultados . . . 14

5. Conclusión **17**
5.1. Resumen del proceso . . . 17
5.2. Principales logros . . . 17

Sprint 3

1. Introducción

1.1. Contexto

El presente proyecto se desarrolla en el marco de la asignatura Tecnologías de Procesamiento Big Data y tiene como objetivo el diseño e implementación de una arquitectura Big Data completa orientada al análisis de datos de trading de criptomonedas. El proyecto simula un entorno real en el que distintas consultoras tecnológicas trabajan para un gran banco que desea crear un nuevo departamento especializado en el asesoramiento a clientes en trading de criptoactivos.

La solución propuesta se basa en tecnologías de Amazon Web Services (AWS) y contempla tanto el procesamiento de datos históricos en modo batch como la ingesta y análisis de datos en tiempo real (streaming). Para la gestión del proyecto se adopta la metodología ágil Scrum, estructurando el desarrollo en sprints incrementales y entregables funcionales al final de cada iteración.

Este documento recoge la planificación y los objetivos correspondientes al **tercer sprint del proyecto**.

1.2. Descripción del problema y objetivos del sprint.

En este sprint se persigue implementar una arquitectura de almacenamiento histórico en tres capas (Bronce, Plata y Oro) y automatizar, mediante AWS Glue y Apache Spark, la lectura de datos CSV desde Bronce, su transformación y persistencia en formato Parquet en Plata, y el cálculo de indicadores técnicos (SMA 200, EMA 50, RSI y MACD) para su almacenamiento final en Oro, con el objetivo de habilitar el análisis técnico de criptomonedas.

Además, se ha modificado la estructura de *buckets* definida en el sprint anterior para adaptarla al esquema Bronce–Plata–Oro.

1.3. Justificación

Este sprint es necesario para organizar y automatizar el tratamiento de la información histórica, garantizando un flujo reproducible desde los datos en bruto hasta los datos listos para análisis. La separación en *tiers* (Bronce, Plata y Oro) facilita la trazabilidad, mejora la calidad del dato y permite desacoplar la ingesta, la transformación y el cálculo de indicadores, haciendo el sistema más mantenible y escalable.

2. Metodología

2.1. Entorno de desarrollo

El inicio de este sprint se centró en adaptar la organización del almacenamiento en Amazon S3, modificando la estructura definida en el sprint anterior a la nueva estructura requerida para el escenario actual. La nueva propuesta mantiene un único *bucket*, pero añade una separación por *tiers* con tres capas: `bronze`, `silver` y `gold`. Dentro de cada capa se conserva la misma organización del sprint anterior, de forma que los datos quedan estructurados como `tier/ada/year/month/archivo`, facilitando la catalogación y el versionado del histórico.

2.2. Diseño de la solución

Para actualizar la estrucutura en los buckets se modifica la clase de **S3YearlyBucketManager**, añadiendo la nueva estrucutura por tiers, como se observa en la imagen.

```
        crypto_name = self._normalize_name(crypto_name)
        for year in range(start_year, end_year + 1):
            for tier in tiers:
                for month in range(1, 13):
                    key = (
                        f"{tier}/"
                        f"{crypto_name}/"
                        f"year={year}/"
                        f"month={month:02d}/.keep"
                    )
                    self.s3.put_object(
                        Bucket=self.bucket_name,
                        Key=key,
                    )
```

Figura 1. Cambio de estructura del *bucket* para soportar los *tiers* (bronze/silver/gold).

Además se ha actualizado el *path* para subir los datos correctamente en las nuevas divisiones en la clase **S3DataUploader**.

```
    def _build_s3_key(self, year: str, month: str, filename: str) -> str:
        return f"{self.tier}/{self.coin}/year={year}/month={month}/{filename}"
```

Figura 2. Cambio de estructura del *path* para soportar los *tiers* (bronze/silver/gold).

Para realizar el paso de las capas, se emplean script ETL jobs de AWS, para el paso *bronze to silver* **BronzeToSilver_ADA**, y para el paso textitbronze to silver y calcular los KPIs, **SilverToGold_KPIs_ADA**

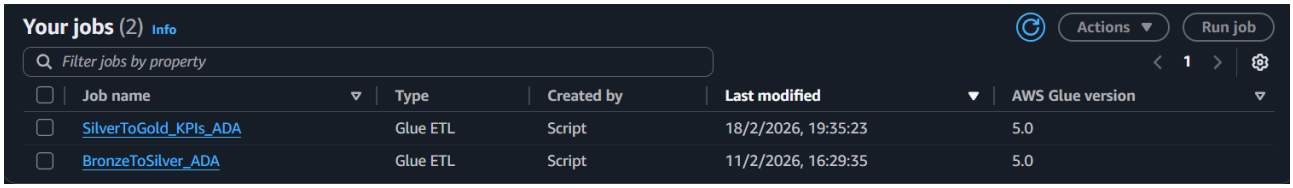


Figura 3. ETL jobs

2.3. Metodología SCRUM

Para seguir la metodología SCRUM, aunque el sprint fuese únicamente de dos semanas, antes de comenzar se analizó la historia de usuario correspondiente al sprint actual:

Historia de Usuario (HU-4)

4.1. Como:
Manager del proyecto.

4.2. Quiero:
Que las consultoras:

- Realicen una implementación del almacenamiento de datos históricos basada en tres capas. Se recomienda que separen estas capas en tres carpetas distintas:
  - Bronce
  - Plata
  - Oro
- Utilicen el servicio AWS Glue y Apache Spark para:
  - Leer los ficheros en formato CSV almacenados en la capa Bronce y guardarlos en la capa Plata en formato Parquet.
  - Calcular los siguientes indicadores (KPIs) que sirvan para realizar un análisis técnico a partir de la información almacenada en la capa Plata:
    - Moving Average Simple (SMA 200)
    - Moving Average Exponential (EMA 50)
    - Relative Strength Index (RSI)
    - Moving Average Convergence Divergence (MACD)
  - Almacenar la información de estos indicadores en la capa Oro.
  - Enlaces sobre indicadores técnicos en trading (como referencia):
    - pablogiltrader.com: Indicadores técnicos trading
    - pablogiltrader.com: Media móvil exponencial
    - pablogiltrader.com: Indicador MACD
    - pablogiltrader.com: RSI trading

4.3. Para:
Realizar un análisis técnico de las criptomonedas.

A partir de la Historia de Usuario se ha realizado un análisis previo para organizar el trabajo del sprint en tareas concretas, asignar responsabilidades dentro del equipo y establecer los criterios de validación necesarios para garantizar que el resultado cumple con los objetivos planteados.

Descomposición en tareas

Tareas del Sprint

ID	Tarea	Asignado a
#18	Documentación del sprint y actualización del informe	Todos
#16	Actualización del *crawler* y *scripts* de subida para la nueva estructura (`bronze/silver/gold`)	Alejandro
#15	Adaptación del cargador/subidor de datos para escribir en Bronce (`bronze/ada/year/month`)	Sergio
#14	Actualización del gestor de *bucket* y prefijos para soportar los tres *tiers*	Liam
#13	Integración del pipeline con AWS Glue: jobs Bronce → Plata y Plata → Oro	Liam
#11	Carga y verificación del histórico en la capa Bronce (CSV)	Todos

3. Implementación

3.1. Arquitectura del pipeline ETL

El pipeline de transformación se implementa mediante AWS Glue Jobs con Apache Spark como motor de procesamiento distribuido. La arquitectura sigue el patrón medallion con tres capas:

- **Bronze:** Datos crudos en CSV.
- **Silver:** Datos limpios en Parquet particionado.
- **Gold:** Datos enriquecidos con KPIs.

Cada transformación entre capas se ejecuta como un job independiente de Glue que orquesta operaciones de Spark, permitiendo escalabilidad y reproducibilidad.

3.2. Procesamiento con Apache Spark: Bronze to Silver

3.2.1. Inicialización del entorno Spark

Todo job de Glue comienza inicializando el contexto de Spark y sus extensiones:

```
sc = SparkContext()                    # Motor de procesamiento
glueContext = GlueContext(sc)          # Extensiones AWS Glue
spark = glueContext.spark_session      # Interfaz SQL
```

SparkContext coordina la ejecución distribuida, **GlueContext** añade capacidades para trabajar con el Data Catalog y DynamicFrames, y **SparkSession** proporciona la interfaz unificada para SQL y DataFrames.

3.2.2. Transformaciones con Spark SQL

El núcleo del procesamiento utiliza Spark SQL para crear un campo timestamp consolidado. Los datos se leen del catálogo como DynamicFrame, se convierten a DataFrame de Spark, y se aplica una transformación SQL:

```
SqlQuery = """
SELECT
    to_timestamp(
        concat(
            cast(year as string), '-',
            lpad(cast(month as string), 2, '0'), '-',
            lpad(cast(day as string), 2, '0'),
            ' 00:00:00'
        )
    ) AS trade_ts,
    symbol, open, high, low, close, volume, day, month, year
FROM myDataSource
"""
```

Operaciones clave de Spark:

- `concat()`: Combina columnas en una sola cadena.
- `lpad()`: Formatea mes/día con ceros a la izquierda (1 → "01").
- `to_timestamp()`: Convierte string a tipo TIMESTAMP de Spark.

La función auxiliar `sparkSqlQuery` gestiona la conversión entre DynamicFrame y DataFrame:

```
def sparkSqlQuery(glueContext, query, mapping, transformation_ctx):
    for alias, frame in mapping.items():
        frame.toDF().createOrReplaceTempView(alias)  # DynamicFrame ->DataFrame
    result = spark.sql(query)                        # Ejecuta SQL
    return DynamicFrame.fromDF(result, glueContext, transformation_ctx)
```

Flujo de ejecución:

1. Convierte DynamicFrame a DataFrame de Spark
2. Registra el DataFrame como vista temporal SQL
3. Ejecuta la query con el optimizador Catalyst de Spark
4. Convierte el resultado de vuelta a DynamicFrame para Glue

3.2.3. Optimizaciones de Spark aplicadas

El pipeline aprovecha características clave de Apache Spark:

- **Lazy evaluation:** Las transformaciones se encolan y Spark optimiza todo el plan de ejecución antes de ejecutarlo.
- **Catalyst optimizer:** Genera automáticamente planes eficientes para las queries SQL, reordenando operaciones y eliminando redundancias.

- **Procesamiento columnar:** Spark trabaja internamente con formato columnar en memoria, optimizando el acceso a datos para queries analíticas.
- **Partition pruning:** Al escribir con `partitionKeys=[ear", "month"]`, consultas futuras solo leerán las particiones necesarias.

3.2.4. Escritura optimizada en Parquet

La salida se escribe en formato Parquet con compresión Snappy y particionamiento:

```
sink = glueContext.getSink(
    path="s3://adaquant-data-bucket/silver/ada/",
    partitionKeys=["year", "month"]
)
sink.setFormat("glueparquet", compression="snappy")
sink.writeFrame(transformed_data)
```

- **Parquet:** Formato columnar que reduce el tamaño 5x respecto a CSV y acelera consultas.
- **Snappy:** Compresión rápida (balance velocidad/ratio).
- **Particionamiento:** Organiza datos en `year=X/month=Y/`, habilitando lectura selectiva.

Ejemplo de mejora de rendimiento:

Operación	CSV sin particiones	Parquet particionado
Leer enero 2024	Lee todo (100 %)	Lee 1 partición (8 %)
Tamaño en disco	500 MB	100 MB

3.3. Ventajas del procesamiento con Spark

- **Escalabilidad horizontal:** Spark distribuye el trabajo en múltiples nodos automáticamente.
- **Expresividad SQL:** Transformaciones complejas se expresan declarativamente.
- **Tolerancia a fallos:** Spark reconstruye particiones perdidas usando el grafo de dependencias (lineage).
- **Procesamiento in-memory:** Mantiene datos en RAM entre operaciones, evitando I/O a disco.

3.4. Procesamiento con Apache Spark: Silver to Gold

3.4.1. Arquitectura distribuida con Window Functions

El job Silver to Gold implementa un procesamiento **totalmente distribuido** utilizando Window Functions de Apache Spark. A diferencia de operaciones agregadas tradicionales, las ventanas permiten calcular indicadores sobre conjuntos ordenados de filas sin romper la paralelización:

```
silver_dyf = glueContext.create_dynamic_frame.from_catalog(
    database="trade_data_imat3a02",
    table_name="silver_ada_ada"
)

df = silver_dyf.toDF()

```

```python
# Ventana base: particionamiento por símbolo
w_base = Window.partitionBy("symbol").orderBy("trade_ts")
df = df.withColumn("_row_num", F.row_number().over(w_base))
```

Ventaja clave de `partitionBy("symbol")`:

Spark distribuye automáticamente los datos de cada criptomoneda a nodos diferentes del cluster. Cada símbolo se procesa en paralelo sin comunicación entre nodos (sin shuffle), permitiendo escalabilidad horizontal ilimitada. En nuestro caso no es necesario porque solo contamos con una criptomoneda pero está implementado por si en un futuro se decidieran añadir nuevas para hacer un estudio de mercado más profundo

Warm-up period: La columna `_row_num` permite validar que existe suficiente historial antes de calcular indicadores, evitando resultados parciales incorrectos.

3.4.2. Cálculo de indicadores técnicos

3.4.2.1. 1. SMA 200 (Simple Moving Average)

```python
w_sma200 = Window.partitionBy("symbol").orderBy("trade_ts") \
                 .rowsBetween(-199, 0)

df = df.withColumn(
    "sma_200",
    F.when(
        F.col("_row_num") >= 200,
        F.avg("close").over(w_sma200)
    ).otherwise(F.lit(None).cast("double"))
)
```

Definición: Promedio móvil simple de los últimos 200 días de precio de cierre.

Fórmula:

$$\text{SMA}_{200}(t) = \frac{1}{200} \sum_{i=t-199}^{t} \text{close}_i$$

Implementación Spark:

- `rowsBetween(-199, 0)`: Define ventana deslizante de 200 filas hacia atrás
- `F.avg(çlose").over()`: Calcula promedio sobre la ventana
- `F.when(_row_num >= 200)`: Solo calcula cuando hay 200 datos válidos

Interpretación:

- Precio >SMA 200 → Tendencia alcista a largo plazo
- Precio <SMA 200 → Tendencia bajista a largo plazo

3.4.2.2. 2. EMA 50 (Exponential Moving Average)

El EMA tradicional es recursivo, lo que rompe la paralelización de Spark. Se implementa una **aproximación EMA truncada** usando ventanas y Higher-Order Functions:

```
K_EMA50 = 300  # Ventana de aproximacion (5-10x el span)

w_ema50 = Window.partitionBy("symbol").orderBy("trade_ts") \
                .rowsBetween(-(K_EMA50 - 1), 0)

df = df.withColumn(
    "ema_50",
    F.when(
        F.col("_row_num") >= 50,
        ema_from_array("_close_hist_50", span=50)
    ).otherwise(F.lit(None).cast("double"))
)
```

Fórmula de aproximación:

$$\text{EMA}_t \approx \frac{\sum_{i=0}^{K-1} \text{decay}^{K-1-i} \cdot x_i}{\sum_{i=0}^{K-1} \text{decay}^{K-1-i}}, \quad \text{decay} = 1 - \alpha, \quad \alpha = \frac{2}{\text{span} + 1}$$

Función auxiliar con aggregate():

```
def ema_from_array(arr_col: str, span: int) -> F.Column:
    alpha = 2.0 / (span + 1.0)
    decay = 1.0 - alpha

    return F.expr(f"""
        aggregate(
            zip_with(
                {arr_col},
                sequence(0, size({arr_col}) - 1),
                (x, i) -> x * pow({decay}, size({arr_col}) - 1 - i)
            ),
            0.0,
            (acc, v) -> acc + v
        ) / aggregate(...)
    """)
```

Operaciones Spark utilizadas:

- `aggregate()`: Función de reducción personalizada de Spark SQL
- `zip_with()`: Combina arrays elemento a elemento
- `sequence()`: Genera secuencia numérica
- `pow(decay, ...)`: Aplica pesos exponenciales decrecientes

Ventaja sobre SMA: Reacciona más rápidamente a cambios de precio al dar mayor peso a valores recientes.

3.4.2.3. 3. RSI 14 (Relative Strength Index)

```
w_rsi = Window.partitionBy("symbol").orderBy("trade_ts") \
              .rowsBetween(-13, 0)
```

```

df = (
    df
    .withColumn("_prev_close", F.lag("close").over(w_base))
    .withColumn("_delta", F.col("close") - F.col("_prev_close"))
    .withColumn("_gain", F.when(F.col("_delta") > 0, F.col("_delta"))
                          .otherwise(0.0))
    .withColumn("_loss", F.when(F.col("_delta") < 0, -F.col("_delta"))
                          .otherwise(0.0))
    .withColumn("_avg_gain", F.avg("_gain").over(w_rsi))
    .withColumn("_avg_loss", F.avg("_loss").over(w_rsi))
    .withColumn(
        "rsi",
        F.when(F.col("_row_num") < 15, None)
         .when(F.col("_avg_loss") == 0, 100.0)
         .otherwise(
             100.0 - (100.0 / (1.0 + F.col("_avg_gain") / F.col("_avg_loss")))
         )
    )
)
```

Definición: Oscilador de momentum que mide la velocidad y magnitud de cambios de precio.

Fórmula:

$$\text{RSI} = 100 - \frac{100}{1 + \text{RS}}, \quad \text{RS} = \frac{\text{Ganancia promedio}}{\text{Pérdida promedio}}$$

Proceso distribuido:

1. `F.lag(çlose")`: Obtiene precio del día anterior usando ventana ordenada
2. Calcula diferencias (deltas) y separa en ganancias y pérdidas
3. `F.avg().over(w_rsi)`: Promedio de 14 períodos usando Window Function
4. Aplica fórmula del Relative Strength

Interpretación:

- RSI $>70 \rightarrow$ Sobrecomprado (posible corrección bajista)
- RSI $<30 \rightarrow$ Sobrevendido (posible rebote alcista)
- RSI $\approx 50 \rightarrow$ Neutral

Nota técnica: Se implementa la versión SMA del RSI (promedio simple de gains/losses) en lugar de la versión Wilder (promedio exponencial recursivo) para mantener la paralelización completa de Spark.

3.4.2.4. 4. MACD (Moving Average Convergence Divergence)

El MACD combina tres componentes calculadas secuencialmente:

```
# 1. Calcular EMA 12 y EMA 26
w_ema12 = Window.partitionBy("symbol").orderBy("trade_ts") \
                .rowsBetween(-(K_EMA12 - 1), 0)
```

```
w_ema26 = Window.partitionBy("symbol").orderBy("trade_ts") \
                .rowsBetween(-(K_EMA26 - 1), 0)

df = df.withColumn("_ema_12", ema_from_array("_close_hist_12", span=12))
df = df.withColumn("_ema_26", ema_from_array("_close_hist_26", span=26))

# 2. MACD Line
df = df.withColumn(
    "macd_line",
    F.when(F.col("_row_num") >= 26, F.col("_ema_12") - F.col("_ema_26"))
     .otherwise(None)
)

# 3. Signal Line (EMA 9 sobre MACD Line)
w_ema9 = Window.partitionBy("symbol").orderBy("trade_ts") \
               .rowsBetween(-(K_EMA9 - 1), 0)

df = df.withColumn(
    "macd_signal",
    F.when(F.col("_row_num") >= 35, ema_from_array("_macd_hist_9", span=9))
     .otherwise(None)
)

# 4. Histogram
df = df.withColumn(
    "macd_histogram",
    F.when(F.col("_row_num") >= 35,
           F.col("macd_line") - F.col("macd_signal"))
     .otherwise(None)
)
```

Componentes:

- **MACD Line:** $\text{EMA}_{12} - \text{EMA}_{26}$
  Mide la divergencia entre tendencia rápida y lenta
- **Signal Line:** $\text{EMA}_9(\text{MACD Line})$
  Suavizado de la MACD Line para generar señales
- **Histogram:** MACD Line − Signal Line
  Visualiza la distancia entre ambas líneas

Señales de trading:

- MACD Line cruza Signal hacia arriba → Señal de compra
- MACD Line cruza Signal hacia abajo → Señal de venta
- Histogram >0 → Momentum alcista creciente
- Histogram <0 → Momentum bajista creciente

3.4.3. Gestión del warm-up period

Cada indicador requiere un mínimo de datos históricos para producir valores válidos:

Indicador	Mínimo (filas)	Razón
SMA 200	200	Necesita 200 precios de cierre
EMA 50	50	Requiere 50 períodos para inicializar
RSI 14	15	15 precios → 14 deltas
MACD Line	26	Depende de EMA 26 (el más lento)
MACD Signal	35	26 (MACD) + 9 (señal)
MACD Hist	35	Depende de Signal Line

El uso sistemático de `F.when(_row_num >= MIN_X)` garantiza que las primeras filas devuelvan `NULL` en lugar de cálculos parciales incorrectos.

3.4.4. Optimizaciones de Spark aplicadas

- **Particionamiento sin shuffle:** Al usar `partitionBy("symbol")`, cada símbolo se procesa en un nodo sin transferencia de datos entre nodos.
- **Window Functions incrementales:** Spark no recalcula toda la ventana en cada fila, sino que mantiene agregados incrementales internos.
- **Lazy evaluation:** Todas las transformaciones (`withColumn`) construyen un grafo de ejecución que Spark optimiza antes de ejecutar.
- **Higher-Order Functions:** Operaciones como `aggregate()` y `zip_with()` se ejecutan nativamente en Catalyst (motor de optimización de Spark SQL).
- **Procesamiento columnar:** Spark trabaja internamente con formato columnar en memoria (similar a Parquet), optimizando accesos a `close`.

3.4.5. Persistencia en capa Gold

Los datos enriquecidos con indicadores se escriben en formato Parquet particionado:

```
df_gold = df.select(
    "trade_ts", "symbol", "open", "high", "low", "close", "volume",
    "day", "month", "year",
    "sma_200", "ema_50", "rsi",
    "macd_line", "macd_signal", "macd_histogram"
)

gold_dyf = DynamicFrame.fromDF(df_gold, glueContext, "gold_dyf")

gold_sink = glueContext.getSink(
    path="s3://adaquant-data-bucket/gold/ada/",
    partitionKeys=["year", "month"]
)
gold_sink.setFormat("glueparquet", compression="snappy")
gold_sink.writeFrame(gold_dyf)
```

La escritura también se ejecuta en paralelo: cada nodo escribe sus particiones directamente a S3 sin coordinación centralizada, aprovechando la escalabilidad de Spark.

3.4.6. Resumen de indicadores calculados

Indicador	Período	Tipo	Propósito
SMA 200	200 días	Tendencia	Dirección a largo plazo
EMA 50	50 días	Tendencia	Tendencia media, reactiva
RSI 14	14 días	Momentum	Sobrecompra/sobreventa
MACD	12/26/9	Momentum	Señales de entrada/salida

Estos KPIs, calculados de forma completamente distribuida con Spark, permiten realizar análisis técnico sobre datos históricos de criptomonedas a escala.

4. Resultados

4.1. Descripción

En esta sección se presentan los resultados obtenidos tras la ejecución del pipeline diseñado en el Sprint 3. Como salida principal, se genera una capa *Gold* en S3 en formato Parquet, particionada por `year` y `month`, que contiene tanto las variables de mercado (`open`, `high`, `low`, `close`, `volume`) como los indicadores técnicos calculados (SMA, EMA, RSI y MACD). Este conjunto de datos sirve como base para análisis histórico, generación de señales y explotación posterior por servicios de consulta o visualización.

Asimismo, la estructura por capas (Bronze $\rightarrow$ Silver $\rightarrow$ Gold) permite mantener trazabilidad del dato, separar responsabilidades (ingesta, limpieza y enriquecimiento) y facilitar la evolución del sistema sin comprometer la consistencia del histórico.

4.2. Ejecución

La ejecución del proceso se realiza mediante un *job* de AWS Glue basado en Apache Spark. A partir de los datos curados en la capa *Silver*, el *job* aplica funciones de ventana (*Window Functions*) para calcular los indicadores de forma distribuida, garantizando que los valores iniciales no se computen con información insuficiente (devolviendo `NULL` cuando no se alcanza el mínimo de filas requerido).

Una vez enriquecidos los registros, el resultado se persiste en la capa *Gold* en formato Parquet con compresión Snappy y particionado temporal. Este enfoque reduce costes de almacenamiento, acelera lecturas selectivas por rango de tiempo y aprovecha la paralelización nativa de Spark para escribir particiones de manera concurrente en S3.

4.3. Discusión de los resultados

Los resultados obtenidos validan la viabilidad de calcular indicadores técnicos a escala sobre series temporales de criptomonedas utilizando un enfoque *serverless* apoyado en Glue y Spark. La persistencia en Parquet particionado permite optimizar tanto el rendimiento de consulta como la mantenibilidad del *data lake*, al tiempo que la separación en capas mejora la calidad del dato y la reproducibilidad de los cálculos.

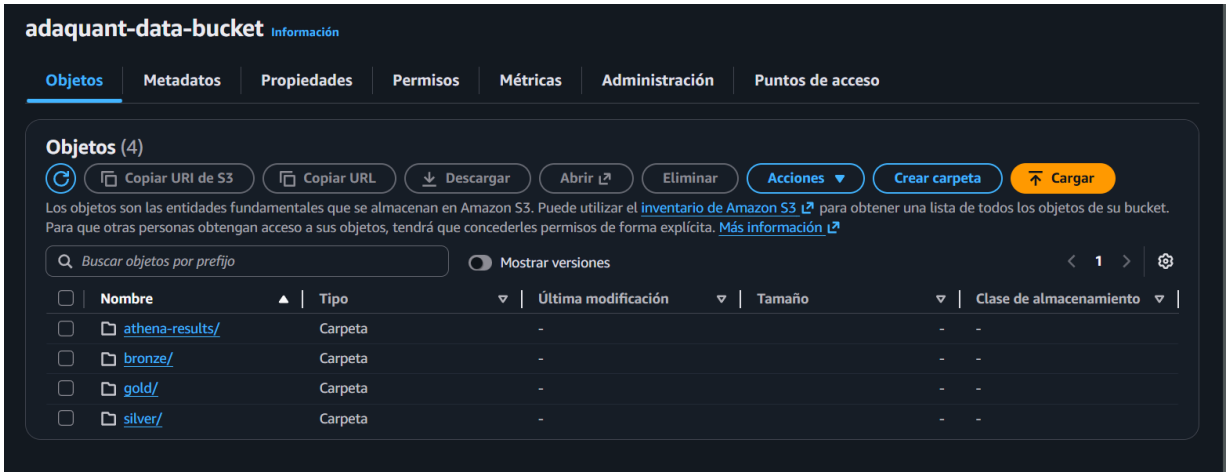


Figura 4. Buckets creados.

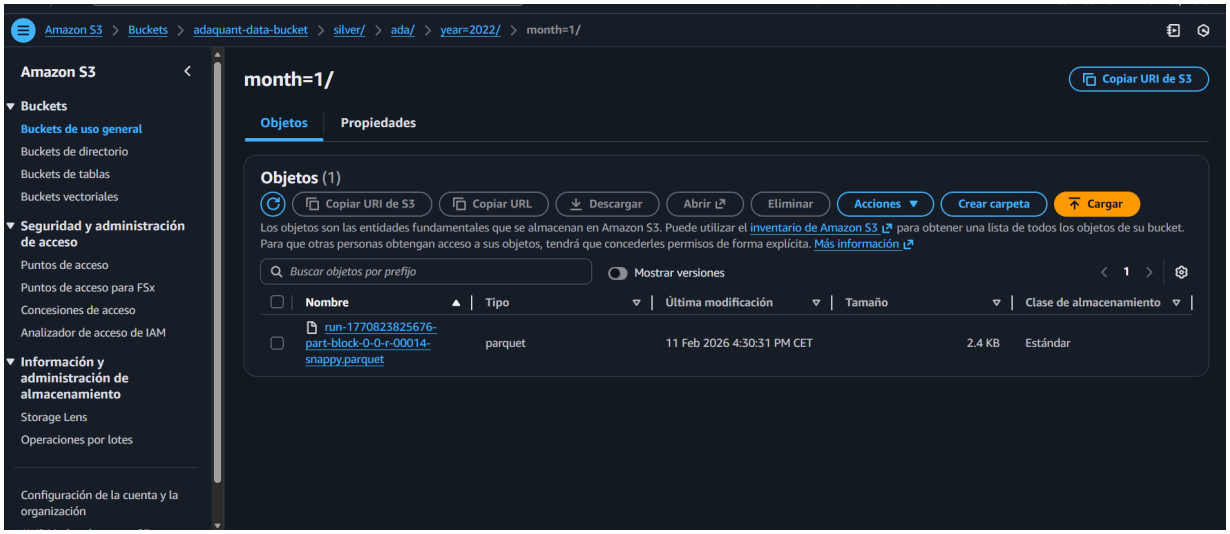


Figura 5. Estructura interna.

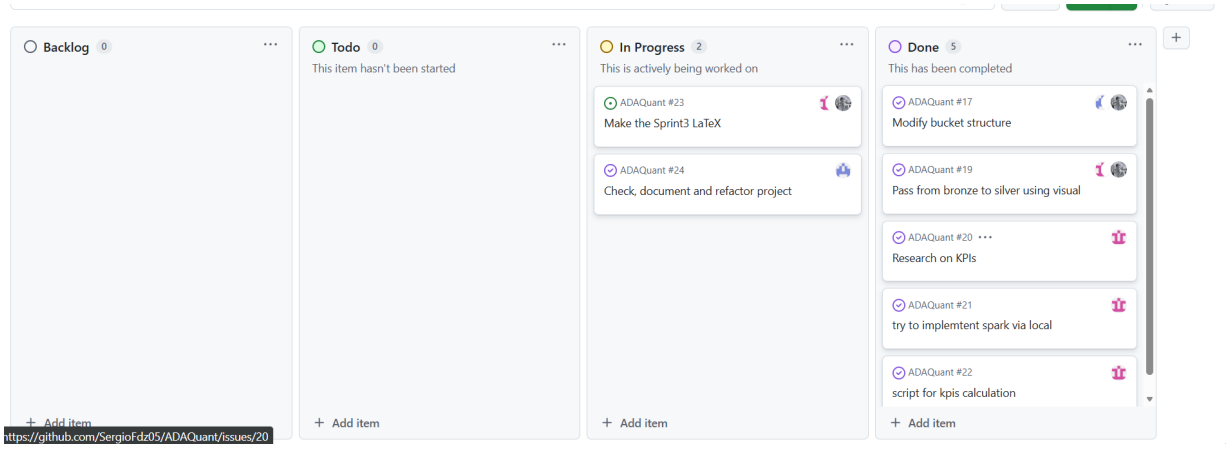


Figura 6. Github board.

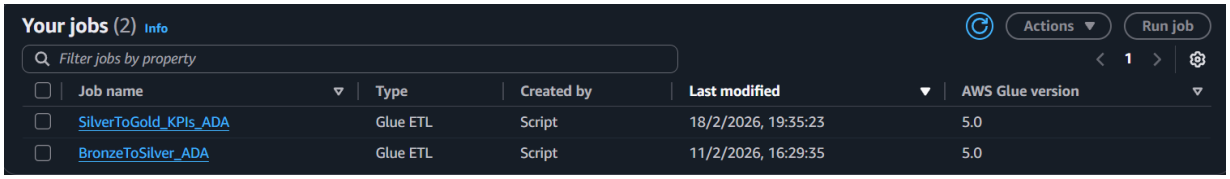
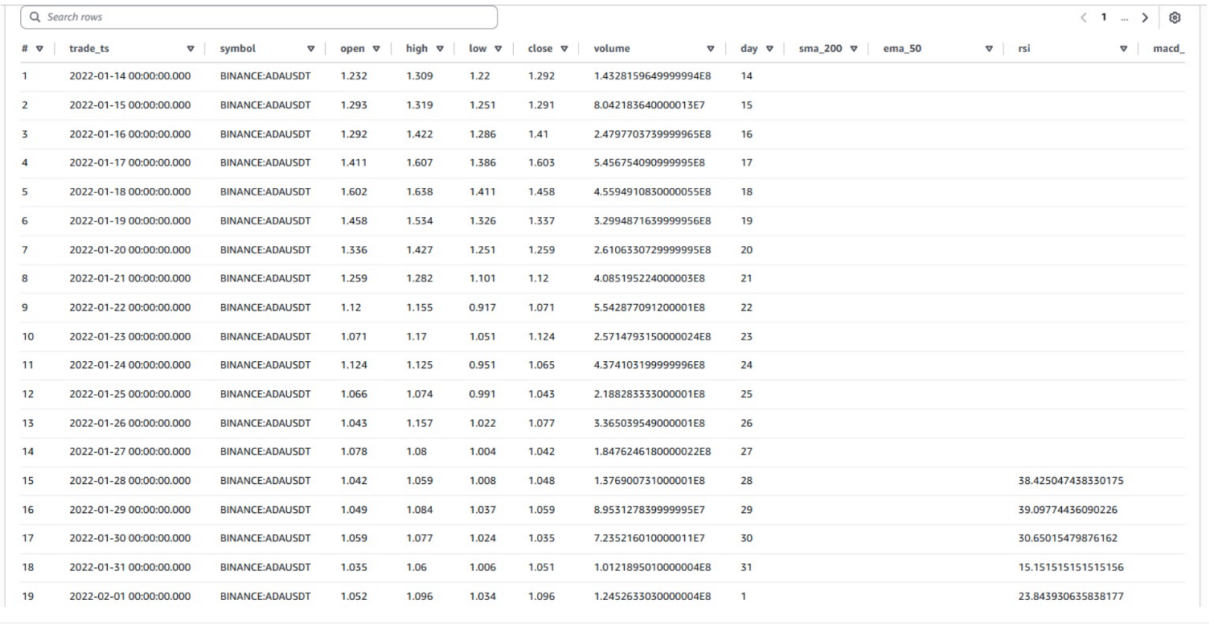


Figura 7. Jobs generados.



Search rows

#	trade_ts	symbol	open	high	low	close	volume	day	sma_200	ema_50	rsi	macd_
1	2022-01-14 00:00:00.000	BINANCE:ADAUSDT	1.232	1.309	1.22	1.292	1.4328159649999994E8	14				
2	2022-01-15 00:00:00.000	BINANCE:ADAUSDT	1.293	1.319	1.251	1.291	8.04218364000013E7	15				
3	2022-01-16 00:00:00.000	BINANCE:ADAUSDT	1.292	1.422	1.286	1.41	2.4797703739999965E8	16				
4	2022-01-17 00:00:00.000	BINANCE:ADAUSDT	1.411	1.607	1.386	1.603	5.456754090999995E8	17				
5	2022-01-18 00:00:00.000	BINANCE:ADAUSDT	1.602	1.638	1.411	1.458	4.5594910830000055E8	18				
6	2022-01-19 00:00:00.000	BINANCE:ADAUSDT	1.458	1.534	1.326	1.337	3.2994871639999956E8	19				
7	2022-01-20 00:00:00.000	BINANCE:ADAUSDT	1.336	1.427	1.251	1.259	2.6106330729999995E8	20				
8	2022-01-21 00:00:00.000	BINANCE:ADAUSDT	1.259	1.282	1.101	1.12	4.085195224000003E8	21				
9	2022-01-22 00:00:00.000	BINANCE:ADAUSDT	1.12	1.155	0.917	1.071	5.542877091200001E8	22				
10	2022-01-23 00:00:00.000	BINANCE:ADAUSDT	1.071	1.17	1.051	1.124	2.5714793150000024E8	23				
11	2022-01-24 00:00:00.000	BINANCE:ADAUSDT	1.124	1.125	0.951	1.065	4.374103199999996E8	24				
12	2022-01-25 00:00:00.000	BINANCE:ADAUSDT	1.066	1.074	0.991	1.043	2.188283333000001E8	25				
13	2022-01-26 00:00:00.000	BINANCE:ADAUSDT	1.043	1.157	1.022	1.077	3.365039549000001E8	26				
14	2022-01-27 00:00:00.000	BINANCE:ADAUSDT	1.078	1.08	1.004	1.042	1.8476246180000022E8	27				
15	2022-01-28 00:00:00.000	BINANCE:ADAUSDT	1.042	1.059	1.008	1.048	1.376900731000001E8	28			38.425047438330175	
16	2022-01-29 00:00:00.000	BINANCE:ADAUSDT	1.049	1.084	1.037	1.059	8.953127839999995E7	29			39.09774436090226	
17	2022-01-30 00:00:00.000	BINANCE:ADAUSDT	1.059	1.077	1.024	1.035	7.235216010000011E7	30			30.65015479876162	
18	2022-01-31 00:00:00.000	BINANCE:ADAUSDT	1.035	1.06	1.006	1.051	1.0121895010000004E8	31			15.151515151515156	
19	2022-02-01 00:00:00.000	BINANCE:ADAUSDT	1.052	1.096	1.034	1.096	1.2452633030000004E8	1			23.843930635838177	

Figura 8. Tabla final de resultados e indicadores calculados.

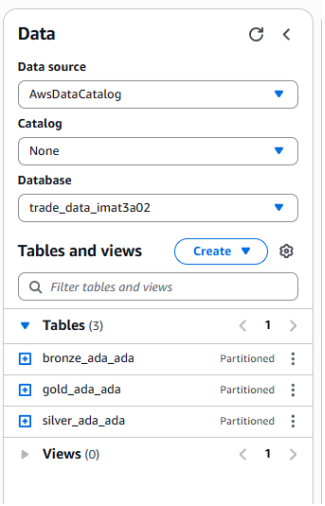


Figura 9. Tablas en Athena.

5. Conclusión

5.1. Resumen del proceso

En este Sprint 3 se ha implementado el almacenamiento histórico en tres capas (Bronze, Silver y Gold) y se ha automatizado el enriquecimiento del dato mediante AWS Glue y Apache Spark. Partiendo de datos limpios en *Silver*, se han calculado indicadores técnicos relevantes y se ha persistido el resultado en *Gold* en formato Parquet, optimizado para consulta y explotación posterior.

5.2. Principales logros

- Definición de una arquitectura de *data lake* por capas con separación clara de responsabilidades.
- Cálculo distribuido de indicadores técnicos (SMA, EMA, RSI y MACD) mediante funciones de ventana en Spark.
- Persistencia eficiente en S3 en formato Parquet, con compresión y particionado temporal para mejorar rendimiento y costes.
- Generación de una base analítica consistente para etapas posteriores de análisis, modelado y visualización.